FRAMEWORK CHOICE

Why Next.js for Production Apps?

لماذا نختار Next.js لتطبيقات الإنتاج؟

Orientation

No code changes

Framework choice, not implementation

LEARNING OUTCOME

Explain why Next.js fits wazifa.app today, how its server/client model works at a preview level, and when another architecture may be better.

We do not choose Next.js because it is popular.

Good engineering starts with product needs, then chooses the simplest reasonable tool for delivery and operation.

The decision is: What does wazifa.app need right now?

UI

React interfaces

Public jobs, admin workflows, forms, and interactive experiences.

SERVER

Trusted server behavior

Validation, authorization, database access, secrets, and provider calls.

DELIVERY

One team, one deployment path

Keep coordination overhead low while the product is still a modular monolith.

Key principle: framework choice should follow requirements and team constraints - not framework loyalty.

Current wazifa.app requirements point toward one full-stack framework.

These needs explain the choice more clearly than any benchmark chart.

Public pages
Server-rendered content and future SEO.

Interactive forms
Authentication and job applications.

Server validation
Distrust browser input and enforce rules.

Authorization
Keep admin and candidate permissions server-owned.

Database + secrets
Private credentials and data access stay off the client.

HTTP endpoints
Explicit contracts for webhooks and external consumers.

Shared product rules
Public and admin experiences evolve together.

One deployment
Reduce cross-service coordination at this stage.

Result: Next.js gives React UI plus useful server capabilities inside one deployment unit.

Think of Next.js as React plus application-level server capabilities.

The browser and server remain different trust boundaries even when the code lives in one repository.

REACT STILL DOES

Components + browser interaction

Components compose pages; interactive pieces manage state, events, and browser-only behavior.

NEXT.JS ADDS

Routing + server execution

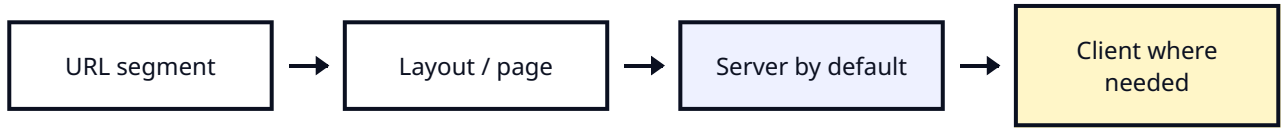
Server rendering, mutations, HTTP endpoints, metadata, and production build conventions.

Important: full-stack in one framework reduces some coordination complexity. It does not erase security, architecture, or runtime boundaries.

Server by default. Client only where interaction needs it.

This is a preview mental model - App Router fundamentals are taught after the first deployment.

APP ROUTER



Better question: what is the smallest part that must run in the browser?

What the App Router gives us

Layouts for shared shells, dynamic routes for job IDs, server-owned data access, and client boundaries only for browser state, events, APIs, and hooks.

Details come later. No directives, file structure, caching APIs, or implementation yet.

Four boundaries, four different jobs.

Server Actions and Route Handlers are different contracts - not competing badges.

01
Server Components
Render on the server, use server-only data through proper layers, pass serializable data across client boundaries.

02
Client Components
Handle browser interaction, state, and hooks. They do not own authorization and must not contain secrets.

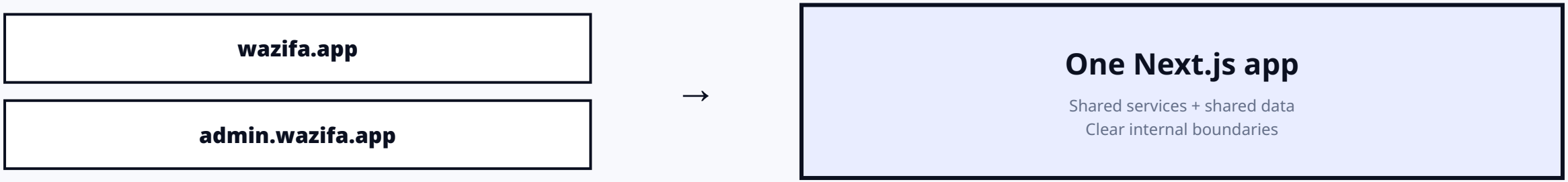
03
Server Actions
Internal server mutations such as create job or submit application. Validation and authorization still apply.

04
Route Handlers
Explicit HTTP endpoints for webhooks, external consumers, or reusable API contracts.

Typical wazifa.app flow: server-owned jobs page → small interactive apply form → protected server mutation → route handler only when an HTTP contract is needed.

Public and admin can begin as one deployment.

One deployment is an architectural choice for the current stage - not permission to mix responsibilities.



Shared types Less duplication	**Shared rules** One product model	**One build** One release unit	**Less network plumbing** At this stage	**Coherent TypeScript** Across UI + server

Later: `proxy.ts` will inspect the host and route public/admin requests. That implementation belongs to Day 4.

TypeScript helps across boundaries - but it is not runtime validation.

Compile-time confidence and runtime trust are different problems.

TYPESCRIPT

Domain shapes across UI + server

Catches many mismatches before deployment and keeps shared contracts coherent.

> **Limit:** browser-submitted values are still untrusted at runtime.

RUNTIME VALIDATION

Real values still need checking

Zod is introduced later when the product needs runtime validation.

> **Rule:** types guide development; validation protects execution.

React 19

UI model

Mongoose

Database integration through repositories

NextAuth / OpenAI / storage SDKs

Providers fit common build conventions

Full-stack in one framework still has costs.

Next.js moves boundaries closer, which makes misunderstanding them easier.

Tradeoff	What it means
Framework complexity	Server and client code can appear close together, so trust boundaries are easy to blur.
Caching + rendering	Defaults and framework versions matter; database libraries may not automatically signal dynamic behavior.
Runtime constraints	Long-running or CPU-heavy work may not fit a request lifecycle; workers may be needed later.
Platform differences	Deployment capabilities and defaults vary by host.
Coupling	Server Actions fit internal UI mutations; Route Handlers provide clearer HTTP contracts.
Upgrade cost	Fast framework evolution requires release-note awareness and build verification.

Reduced complexity is not deleted complexity.

Sometimes a separate backend is the better choice.

Architecture should match current requirements, team capability, and expected change.

SEPARATE BACKEND MAY FIT

- Multiple independent clients need a stable shared API.
- Frontend and backend teams deploy independently.
- Long-running jobs or specialized runtimes dominate the workload.
- Compliance or organizational boundaries require separation.
- Existing domain services already own data and business rules.

ANOTHER FRONTEND MAY FIT

- The team has stronger expertise elsewhere.
- The product is primarily static.
- The workload is native mobile or desktop.
- An existing architecture already solves the product constraints better.

> **Evolution is allowed:** Next.js can still consume a separate backend later.

Map product behavior to the boundary it actually needs.

This is the course roadmap preview - not implementation.

Job listing + detail

Read data on the server; later expose search-friendly metadata.

Login + apply forms

Focused client interaction; authorization remains server-side.

Create job + apply

Internal Server Action workflows.

Callbacks + webhooks

Explicit HTTP endpoints where the contract requires them.

Public + admin hosts

One deployment routed through `proxy.ts` later.

Database access

Mongoose stays behind repository boundaries.

AI screening

Starts in request flow; worker only when runtime pain justifies it.

Vertical slices

Page, server behavior, validation, and deployment evolve together.

Course rule: Next.js does not replace the service/repository boundaries used by wazifa.app.

Can you defend the choice without overselling it?

Lecture 3 is complete when you can explain both the fit and the tradeoffs.

- [ ] List the wazifa.app requirements that motivated Next.js.
- [ ] Explain the one-deployment benefit and its limits.
- [ ] Name cases where a separate backend or another framework is better.
- [ ] Distinguish Server Components, Client Components, Server Actions, and Route Handlers.
- [ ] Explain why TypeScript is not runtime validation.
- [ ] Avoid teaching file structure, directives, caching APIs, or commands early.

01

Choose from product requirements, not framework loyalty.

02

Keep the client boundary as small as interaction requires.

03

One deployment lowers coordination overhead - boundaries still matter.

NEXT LECTURE

Lecture 4 - Course Method: Spiral + Ship →